The FAULT framework for Failure Analysis and Understanding for Language-to-SPARQL Testing

Indeewari Balasooriya[1,*], Bianca Benelli[1], Niccolò Marastoni[1], Elisa Quintarelli[1] and Matteo Lissandrini[1]

[1]University of Verona, Verona, Italy

Abstract

Natural-language-to-SPARQL (NL2SPARQL) systems are commonly evaluated using a single aggregate accuracy score, which obscures distinct sources of failure, such as noisy phrasing, query complexity, schema-grounding errors, and reliance on lexicalized IRIs. We present FAULT, a diagnostic framework for enriching NL2SPARQL benchmarks with metadata along three axes: computational question type, SPARQL structural features, and linguistic/semantic noise. FAULT combines automatic query analysis with manual noise annotation and is supported by an open-source, semi-automated annotation toolchain. We apply the framework to two recent benchmark datasets from the Text2SPARQL'25: a toy graph, CorporateKG, and a DBpedia snapshot, producing 95 and 121 annotated query resources, respectively. Through an illustrative evaluation of the SPARQL-LLM method, we show how FAULT exposes hidden performance factors, including sensitivity to noise in the questions, dependence on in-context examples, and reliance on readable IRI strings. Hence, FAULT provides reusable resources and tooling to enable more comprehensive diagnostic evaluation of NL2SPARQL systems.

Keywords

NL2SPARQL, Knowledge Graph Question Answering, Benchmark Resource, Failure Analysis, Error Taxonomy

1. Introduction

Translating natural questions into SPARQL remains a central challenge in Knowledge Graph Question Answering (KGQA). A sequence of benchmarks (namely: QALD [1, 2], LC-QuAD and LC-QuAD 2.0 [3, 4], SciQA [5], DBLP-QuAD [6], Spider4SPARQL [7], and CK25 from the Text2SPARQL challenge [8]) have enabled comparative evaluation of NL2SPARQL systems, and many neural and LLM-based approaches have been evaluated against them [9, 10, 11, 12, 13]. However, these efforts consistently report a single aggregate execution metric (e.g., exact match or F1 over result bindings), which obscures *why* a system succeeds or fails on a given instance: whether the cause is sensitivity to surface-level phrasing, the structural complexity of the target query, or an inability to correctly ground language onto the knowledge graph's schema and identifiers. Studies on Text-to-SQL robustness [14, 15, 16, 17] show that aggregate accuracy in tabular question answering is similarly confounded by annotation errors and surface level variation, and a parallel line of work has begun to audit KGQA benchmarks and retrieval-augmented pipelines for comparable pitfalls [18], while large scale annotated resources such as QueryBridge [19] aim to broaden the pool of question-query pairs available for training and evaluation. To date, there is no systematic framework to analyze the characteristics of a benchmark and identify how a given system is affected by them. To address this gap, we introduce **FAULT** (Failure Analysis and Understanding for Language-to-SPARQL Testing), a framework for augmenting NL2SPARQL benchmarks with diagnostic metadata. FAULT is complementary to existing benchmark efforts: rather than creating another benchmark, it provides a reusable *annotation methodology* and toolchain that can be layered onto existing and future datasets, to support fine-grained error analysis and make sources of failure explicit and analyzable. The framework combines: (1) Computational question type annotations, grounded in SPARQL operator usage rather than surface linguistic form; (2) Structural feature extraction, building on the LSQ vocabulary [20]; (3) Linguistic and semantic noise annotations, encoded with the

ISWC 2026 Companion Volume, October 25–29, 2026, Bari, Italy

*Corresponding author.

indeewari.balasooriya@univr.it (I. Balasooriya); bianca.benelli@univr.it (B. Benelli); niccolo.marastoni@univr.it (N. Marastoni); elisa.quintarelli@univr.it (E. Quintarelli); matteo.lissandrini@univr.it (M. Lissandrini)



© 2026 Copyright for this paper by its authors. Use permitted under Creative Commons License Attribution 4.0 International (CC BY 4.0).

Qanary/QA vocabulary [21] and the Web Annotation ontology; (4) Tooling for benchmark enrichment and analysis, released as open-source software.

To showcase our work, we instantiate FAULT on the datasets from the Text2SPARQL'25 challenge. This initial scope allows us to validate the annotation methodology and toolchain end-to-end, while releasing a reusable seed resource for future diagnostic studies. In particular, we are the first to quantify the confounding effect of lexicalized IRIs in existing benchmarks. Future work will extend the analysis to additional benchmarks and systems and use it to systematically re-evaluate existing systems.

2. The FAULT Framework

FAULT characterizes each question-query pair along three axes: question type, SPARQL structural features, and linguistic/semantic noise, which are analyzed via a semi-automatic annotation workflow.

2.1. FAULT characterization

The first two closely related dimensions for analysis investigate how a given system performs over some specific subset of questions and with specific SPARQL operators. Question types form a two-level hierarchy grounded in *observable SPARQL constructs* rather than surface linguistic form (e.g., "wh-question" vs. "yes/no"), so that category assignment is automatable and reproducible. The root class distinguishes **Confirmation** (ASK queries) from **Factoid** (SELECT queries). Factoid is further specialized into seven subtypes: **Enumeration** (SELECT DISTINCT for plain enumerations, or GROUP BY for aggregate enumerations, both ensuring duplicate-free results), **Ranked Listing** (ORDER BY over an existing property, no cardinality bound) and its subtype **Limited Ranked Listing** (ORDER BY+LIMIT, e.g. "top-k"), **Aggregation** (COUNT/SUM/AVG/MAX/MIN with no grouping), **Sampling** (unordered, cardinality-restricted result set, via SAMPLE or an unranked LIMIT without ORDER BY), **Counterfactual Identification** (FILTER NOT EXISTS, identifying entities that lack a given property or relationship), **Aggregate Enumeration** (inheriting from both Enumeration and Aggregation: an enumeration of group/aggregate-value pairs, e.g., "average salary per department," mapped to GROUP BY with an aggregate projection), and **Comparative** (also inheriting from both Enumeration and Aggregation: computes an aggregate across two or more groups and returns the entity or entities at the extreme aggregate value, mapped to GROUP BY with a subquery-based FILTER on MAX/MIN that avoids the unsafe ORDER BY + LIMIT 1 pattern, which silently discards ties). An unspecialized Factoid label is retained for single binding SELECT queries that trigger none of these seven specializations. We treat SubQuery usage as an orthogonal *structural* feature rather than a question type, since it can occur within several categories (e.g., a tie-safe Ranked Listing, or a Comparative query computing an aggregate threshold). Our framework analyzes ground truth queries automatically annotating them with features from the LSQ vocabulary [20]: operator usage, triple patterns and BGP counts, and subquery usage; these features drive both automatic question type assignment. Further, to ensure the quality of a given benchmark dataset, we exploit this mechanism and propose a tool to detect anti-patterns in ground truth queries (as described in Section 2.2).

Question Noise. The second dimension of analysis is the ability of the system to interpret questions that contain some form of linguistic or semantic noise. Hence, FAULT separates failures attributable to *how* a question is phrased from failures attributable to *what* it requires the system to resolve against the schema. **Linguistic** issues affect surface form without changing question intent, these include: lexical grounding mismatch (a schema term synonym, e.g., "telephone" for phone number), orthographic noise, abbreviations, syntactic distortion, and discourse reference. **Semantic** issues affect what must be inferred or resolved, independent of phrasing. We distinguish: implicit relation inference (an unstated multi-hop path), entity/attribute mismatch, aggregation underspecification, entity linking ambiguity, logical form ambiguity, vague predicate grounding (e.g., "young employees", no explicit threshold), and schema coverage mismatch. Two subtypes, schema coverage mismatch and vague predicate grounding,

describe cases that no system could resolve from question, schema, and KG alone; we flag these separately so downstream analyses can exclude intrinsically unanswerable questions when estimating a practical performance ceiling.

Reliance on Lexicalized IRIs. Existing benchmarks commonly feature KGs with lexicalized IRIs, e.g., in the Corporate KG graph (CK25) [8] the department entity is exposed through a readable identifier such as `pv:Department`. When analyzing LLM-based systems performance, the presence of lexicalized IRIs acts as a confounder since it is hard to identify whether the system correctly interpreted the question and the KG schema or simply "*guessed*" which IRI to use. For this reason, we propose to obfuscate the IRIs in the benchmark with their hashed version.

The transformation is deterministic and namespace-aware. We hash all IRIs whose namespace belongs to a predefined set of benchmark-specific prefixes, such as the CK25 and DBpedia prefixes, while leaving literals and external vocabulary IRIs, such as RDF, RDFS, OWL, and XSD terms, unchanged. During hashing, the namespace prefix is preserved, and only the local name is replaced. This obfuscation removes lexical cues from identifiers while keeping the benchmark executable and structurally equivalent to the original. Importantly, labels, comments, literal values, and other textual metadata are not hashed, so the experiment distinguishes reliance on lexicalized IRIs from legitimate use of human-readable schema descriptions.

This obfuscation is obtained by hashing with SHA-256 the local name of a IRI, reduced modulo 10^k, then zero-padded to k digits (default $k = 6$). In addition, the generated mapping is monitored for potential collisions; if a potential collision is detected, the hash length can be increased, or the mapping regenerated with a larger namespace-specific code space. To monitor the risk of collisions, we estimate the number of unique IRIs per namespace using a HyperLogLog [22] sketch and issue a warning when the estimate approaches the available hash space. Since the transformation is deterministic and cached, the same original IRI is mapped to the same hashed identifier across files and runs. The same mapping is applied consistently across all benchmark artifacts, including Turtle data, SPARQL queries, SHACL shapes, and IRIs embedded in SPARQL strings.

2.2. Annotation Workflow

FAULT proposes to annotate the pairs of NL questions and corresponding SPARQL queries in the ground truth via a multi-step annotation workflow. The workflow combines automatic query analysis with manual semantic annotation. We developed the *SPARQL Annotator*, an automatic tool that parses each ground truth query with RDFLib, extracts structural features and vocabulary terms, and flags potential query-level issues for review, such as anti-patterns, operator mismatches, unsafe pagination, or syntax errors (e.g., misplaced `DISTINCT` clauses). These features are then used to automatically derive question-type labels. Linguistic and semantic noise is annotated manually by multiple annotators through a web-based tool following a shared protocol. As common in this case, we propose to resolve disagreements via voting. Our experience showed that this can be better achieved after a trial run and discussion about the interpretation of the categories to ensure consistency across raters.

For each noisy question, FAULT can also provide a clean natural language question counterpart linked via `skos:closeMatch`. These remove ambiguity and align with the schema and reference SPARQL query while preserving the original intent. Linking each noisy–clean pair enables controlled evaluation of system robustness to input variation.

Figure 1 shows the resulting annotation graph for one CK25 pair. The noisy question is typed with an entity linking ambiguity noise class via the Qanary/QA vocabulary [21] ("Ms. Brant" may refer to more than one employee in the target KG); the clean counterpart carries LSQ style structural features and a question type label. We annotate noise *relative to the target KG*, e.g., the pair in the figure is ambiguous because the KG instance data contains more than one matching entity, independent of whether the phrasing would be natural in a multi-turn conversation.

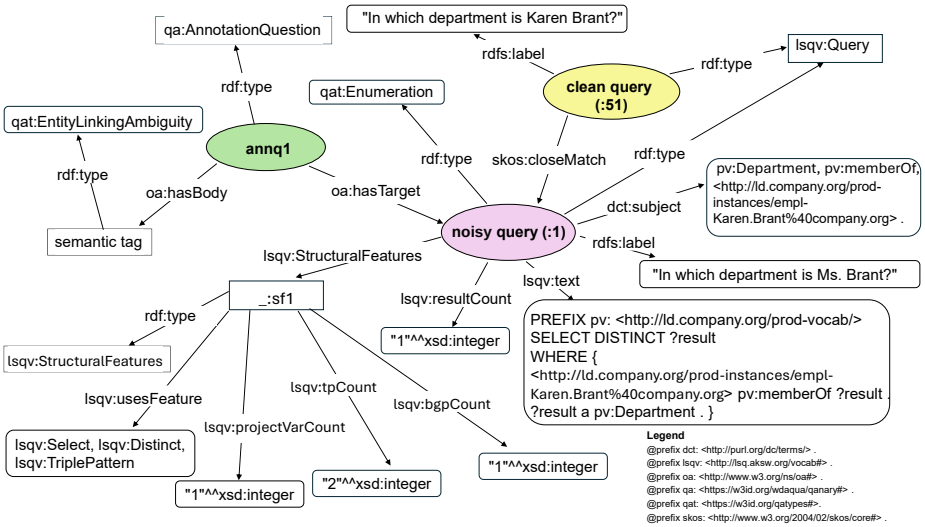


Figure 1: Annotation for the noisy clean question pair *"In which department is Ms. Brant?"* (noisy query :1) and *"In which department is Karen Brant?"* (clean query :51).

3. Illustrative Evaluation

To demonstrate FAULT's diagnostic value, we evaluated SPARQL-LLM [13], a retrieval-augmented system grounding generation in VoID [23] schema descriptions and query text exemplars over both datasets (CK25 with 50 questions and DBpedia with 100 questions) in the Text2SPARQL'25 challenge. Their main features are reported in Table 1.

For each dataset, we test the effect of *IRI lexicalization*, comparing both the original and hashed versions, and for each version, we investigate the performance across the questions affected by either linguistic or semantic noise (16 linguistic only, 23 semantic only, 6 mixed; DBpedia: 12, 8, 1, respectively). Further, since the method exploits *in-context examples* we test the effect of those as well (With vs. Without). We report instance-level F1 score over result bindings (Table 2). Due to space limitations, below, we analyze CK25 numbers, as shown in the table, DBpedia's analysis is analogous.

Lexicalized IRIs dominate. Hashing IRIs drops F1 from 0.487 to 0.263 on clean questions with examples, showing that much of SPARQL-LLM's apparent accuracy reflects lexical overlap with IRI strings rather than schema understanding.

Examples help independently of IRI legibility: F1 improves on both original and clean variants when examples are supplied, increasing from 0.243 to 0.295 with lexicalized IRIs and from 0.022 to 0.215 with hashed IRIs on original questions, while clean reformulations show the same trend (0.462 to 0.487 and 0.015 to 0.263), confirming genuine structural guidance beyond surface form.

Noise layers are not interchangeable: the presence of linguistic and semantic noise reduces F1 by 19.2 points in the best configuration (with examples and non-hashed IRIs).

Questions with only linguistic noise are generally less affected than those with semantic noise. This suggests that surface-level variations, such as lexical mismatches or syntactic distortions, are easier

Table 1
Knowledge Graph Statistics

Property	Corporate KG	DBpedia
Distinct Classes	20	752
Distinct Predicates	50	61855
Distinct Subjects	2693	14309435
Distinct Objects	8910	34985051
Triples	27035	173683188

Dataset	Questions	With Examples		Without Examples	
		Not Hashed	Hashed	Not Hashed	Hashed
CK25	Original (50)	0.295	0.215	0.243	0.022
	Linguistic only (16)	0.274	0.207	0.216	0.017
	Semantic only (23)	0.185	0.166	0.157	0.014
	Mixed (6)	0.004	0.002	0.025	0.000
	Clean reformulation (45)	0.487	0.263	0.462	0.015
DBpedia	Original (100)	0.626	0.137	0.284	0.006
	Linguistic only (12)	0.671	0.117	0.137	0.039
	Semantic only (8)	0.310	0.129	0.123	0.000
	Mixed (1)	0.000	0.000	0.000	0.000
	Clean reformulation (21)	0.627	0.143	0.264	0.051

Table 2
SPARQL-LLM F1 across configurations, CK25 (Corporate) and DBpedia (large-KG) tracks of Text2SPARQL'25. "Original" denotes the (possibly) noisy benchmark questions. "Linguistic/Semantic only" includes questions affected exclusively by the corresponding noise layer. "Mixed" are questions containing both linguistic and semantic noise. "Clean reformulation" denotes noise-free reformulations of the corresponding original noisy questions. Numbers in parentheses indicate the number of questions.

for the system to handle than cases requiring deeper schema reasoning. In particular, semantic noise is often associated with implicit relation inference and entity/attribute mismatch, which may require discovering multi-hop paths in the KG. Questions that combine linguistic and semantic noise are the most challenging, with F1 dropping almost to zero (0.0038).

Without examples and with hashed IRIs, F1 collapses to 0.000–0.022, exposing a substantial symbolic reasoning gap when lexical and example-based shortcuts are both removed.

4. Limitations, Availability, and Roadmap

We have tested here FAULT on two datasets (CK25, 50 questions; DB25, 100 questions) against a single system, a scoping choice for validating the methodology end-to-end on a hand checked corpus, not a claim of generality. In the future, we plan to: (i) port FAULT to further benchmarks (QALD, LC-QuAD 2.0, SciQA, DBLP-QuAD, and large-scale resources such as QueryBridge [19]) to test taxonomy portability; (ii) evaluate additional NL2SPARQL systems beyond SPARQL-LLM to separate system-specific from general findings; (iii) refine our noise taxonomy based also on inter annotator agreement from a larger multi-annotator noise study of more benchmarks; and (iv) connect FAULT to related benchmark-auditing efforts [18]. Further, we are developing a new dataset based on GPTKB [24]. GPTKB is a general-domain KB materialized entirely from an LLM (GPT-4.1; 100M+ triples, 6.1M entities). We are curating a complementary, fully disambiguated zero-noise benchmark over GPTKB as a controlled counterpart to CK25's noise. One dimension we want to add to FAULT is schema complexity, and with GPTKB, we aim to produce a controlled test case for that dimension as well.

FAULT and all resources are publicly available:

- ontology at https://w3id.org/qatypes;
- SPARQL Annotator at https://github.com/UniVR-DH/sparql-annotator;
- annotation tool at https://github.com/niccolomarastoni/nl-to-sparql_query_annotation;
- data and scripts at https://github.com/UniVR-DH/nl2sparql-benchmark;
- the archived release on Zenodo at https://doi.org/10.5281/zenodo.20074201.

Acknowledgements

This work has been partially supported by DataGEMS, funded by the European Union's Horizon Europe Research and Innovation programme, under grant agreement No 101188416.

Declaration on Generative AI

During the preparation of this work, the authors used Grammarly to check grammar, spelling, and readability. ChatGPT (OpenAI) and Claude (Anthropic) were used to assist with manuscript wording and editing. GitHub Copilot was used to support code-related problem solving, debugging, and code review. All AI-assisted outputs were reviewed and validated by the authors, who take full responsibility for the accuracy, integrity, and final content of the manuscript and accompanying software resources.

References

[1] R. Usbeck, X. Yan, A. Perevalov, L. Jiang, J. Schulz, A. Kraft, C. Möller, J. Huang, J. Reineke, A.-C. Ngonga Ngomo, et al., QALD-10 – the 10th challenge on question answering over linked data: Shifting from DBpedia to Wikidata as a KG for KGQA, Semantic Web 15 (2024) 2193–2207.

[2] A. Perevalov, D. Diefenbach, R. Usbeck, A. Both, Qald-9-plus: A multilingual dataset for question answering over dbpedia and wikidata translated by native speakers, in: 2022 IEEE 16th International Conference on Semantic Computing (ICSC), IEEE, 2022, pp. 229–234.

[3] P. Trivedi, G. Maheshwari, M. Dubey, J. Lehmann, LC-QuAD: A corpus for complex question answering over knowledge graphs, in: The Semantic Web – ISWC 2017: 16th International Semantic Web Conference, Vienna, Austria, October 21–25, 2017, Proceedings, Part II, Springer-Verlag, Berlin, Heidelberg, 2017, pp. 210–218. URL: https://doi.org/10.1007/978-3-319-68204-4_22. doi:10.1007/978-3-319-68204-4_22.

[4] M. Dubey, D. Banerjee, A. Abdelkawi, J. Lehmann, LC-QuAD 2.0: A large dataset for complex question answering over Wikidata and DBpedia, in: International Workshop on the Semantic Web, 2019. URL: https://api.semanticscholar.org/CorpusID:198166992.

[5] S. Auer, D. A. C. Barone, C. Bartz, E. Cortes, M. Y. Jaradeh, O. Karras, M. Koubarakis, D. I. Mouromtsev, D. Pliukhin, D. Radyush, I. Shilin, M. Stocker, E. Tsalapati, The SciQA scientific question answering benchmark for scholarly knowledge, Scientific Reports 13 (2023). URL: https://api.semanticscholar.org/CorpusID:258507546.

[6] D. Banerjee, S. Awale, R. Usbeck, C. Biemann, Dblp-quad: A question answering dataset over the DBLP scholarly knowledge graph, in: I. Frommholz, P. Mayr, G. Cabanac, S. Verberne, J. Brennan (Eds.), Proceedings of the 13th International Workshop on Bibliometric-enhanced Information Retrieval co-located with 45th European Conference on Information Retrieval (ECIR 2023), Dublin, Ireland, April 2nd, 2023, volume 3617 of *CEUR Workshop Proceedings*, CEUR-WS.org, 2023, pp. 37–51. URL: https://ceur-ws.org/Vol-3617/paper-05.pdf.

[7] C. Kosten, P. Cudré-Mauroux, K. Stockinger, Spider4SPARQL: A complex benchmark for evaluating knowledge graph question answering systems, in: 2023 IEEE International Conference on Big Data (BigData), IEEE, 2023, pp. 5272–5281. URL: http://dx.doi.org/10.1109/BigData59044.2023.10386182. doi:10.1109/bigdata59044.2023.10386182.

[8] eccenca GmbH, CK25: Corporate knowledge reference dataset for benchmarking Text2SPARQL question answering approaches, https://github.com/eccenca/ck25-dataset, 2025.

[9] P. A. K. K. Diallo, S. Reyd, A. Zouaq, A comprehensive evaluation of neural sparql query generation from natural language questions, IEEE Access 12 (2024) 125057–125078.

[10] P. A. K. K. Diallo, A. Zouaq, FRASE: frame-based structured representations for generalizable SPARQL query generation, in: S. Piperidis, N. Bel, H. van den Heuvel, N. Ide, S. Krek, A. Toral (Eds.), Proceedings of the Fifteenth Language Resources and Evaluation Conference, LREC 2026, Palma de Mallorca, Spain, May 13-15, 2026, ELRA Language Resource Association / ACL, 2026, pp. 1303–1319. URL: https://doi.org/10.63317/52g4z7jtim8o. doi:10.63317/52G4Z7JTIM8O.

[11] A. Gashkov, A. Perevalov, M. Eltsova, A. Both, SPARQL query generation with llms: Measuring the impact of training data memorization and knowledge injection, in: H. Verma, A. Bozzon, A. Mauri, J. Yang (Eds.), Web Engineering - 25th International Conference, ICWE 2025, Delft, The Netherlands, June 30 - July 3, 2025, Proceedings, volume 15749 of *Lecture Notes in Computer Science*,

Springer, 2025, pp. 177–192. URL: https://doi.org/10.1007/978-3-031-97207-2_14. doi:10.1007/978-3-031-97207-2_14.

[12] S. Reyd, A. Zouaq, Assessing the generalization capabilities of neural machine translation models for SPARQL query generation, in: The Semantic Web – ISWC 2023: 22nd International Semantic Web Conference, Athens, Greece, November 6–10, 2023, Proceedings, Part I, Springer-Verlag, Berlin, Heidelberg, 2023, pp. 484–501. URL: https://doi.org/10.1007/978-3-031-47240-4_26. doi:10.1007/978-3-031-47240-4_26.

[13] P. Smeros, V. Emonet, R. Wang, A.-C. Sima, T. M. de Farias, SPARQL-LLM: Real-time SPARQL query generation from natural language questions, 2025. URL: https://arxiv.org/abs/2512.14277. arXiv:2512.14277.

[14] S. Chang, J. Wang, M. Dong, L. Pan, H. Zhu, A. H. Li, W. Lan, S. Zhang, J. Jiang, J. Lilien, S. Ash, W. Y. Wang, Z. Wang, V. Castelli, P. Ng, B. Xiang, Dr.spider: A diagnostic evaluation benchmark towards text-to-sql robustness, in: The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023, OpenReview.net, 2023. URL: https://openreview.net/forum?id=Wc5bmZZU9cy.

[15] L. Tu, R. Wang, T. Sheng, S. Ravi, D. Roth, LLM NL2SQL robustness: Surface noise vs. linguistic variation in traditional and agentic settings, 2026. URL: https://arxiv.org/abs/2603.17017. arXiv:2603.17017.

[16] T. Jin, Y. Choi, Y. Zhu, D. Kang, Text-to-SQL benchmarks are broken: An in-depth analysis of annotation errors, in: Conference on Innovative Data Systems Research, 2026. URL: https://api.semanticscholar.org/CorpusID:284170152.

[17] M. Safarzadeh, A. Oroojlooy, D. Roth, Evaluating NL2SQL via SQL2NL, in: Findings of the Association for Computational Linguistics: EMNLP 2025, Association for Computational Linguistics, Suzhou, China, 2025, pp. 18954–18968.

[18] L. Zhang, Z. Jiang, H. Chi, H. Chen, M. Elkoumy, F. Wang, Q. Wu, Z. Zhou, S. Pan, S. Wang, et al., Diagnosing and addressing pitfalls in kg-rag datasets: Toward more reliable benchmarking, Advances in Neural Information Processing Systems 38 (2026).

[19] A. Orogat, A. El-Roby, QueryBridge: One million annotated questions with SPARQL queries - dataset for question answering over knowledge graphs, in: Proceedings of the 34th ACM International Conference on Information and Knowledge Management, CIKM '25, Association for Computing Machinery, New York, NY, USA, 2025, pp. 6503–6507. URL: https://doi.org/10.1145/3746252.3761623. doi:10.1145/3746252.3761623.

[20] C. Stadler, M. Saleem, A.-C. N. Ngomo, Lsq framework: The lsq framework for sparql query log processing., in: QuWeDa@ ISWC, 2022, pp. 49–64.

[21] K. Singh, A. Both, D. Diefenbach, S. Shekarpour, Towards a message-driven vocabulary for promoting the interoperability of question answering systems, in: 2016 IEEE Tenth International Conference on Semantic Computing (ICSC), 2016, pp. 386–389. doi:10.1109/ICSC.2016.59.

[22] A. Nazi, B. Ding, V. R. Narasayya, S. Chaudhuri, Efficient estimation of inclusion coefficient using hyperloglog sketches, Proc. VLDB Endow. 11 (2018) 1097–1109. URL: http://www.vldb.org/pvldb/vol11/p1097-nazi.pdf. doi:10.14778/3231751.3231759.

[23] W3C, Describing linked datasets with the VoID vocabulary, 2011. URL: https://www.w3.org/TR/void/.

[24] Y. Hu, T.-P. Nguyen, S. Ghosh, M. Müller, S. Razniewski, GPTKB v1.5: A massive knowledge base for exploring factual LLM knowledge, Proceedings of the AAAI Conference on Artificial Intelligence 40 (2026) 41604–41606. URL: http://dx.doi.org/10.1609/aaai.v40i48.42354. doi:10.1609/aaai.v40i48.42354.